Data Science

Class 1

Process Basics

- → Numpy, pandas, Scikit Learn
- → Data Exploration
- → Data Cleaning
- → Model Evaluation

Core Algorithms

Regression — Linear Reg, Logistic Reg.

Classification — Decision Trees, K-Nearest Neighbours, SVM, Random Forest, Ensemble Tech.

Clustering — K-Means Clustering, DBSCAN

Reinforcement Learning

Lab work

1. Assignments
2. Final Project

Problem Statement

Figure out whether a candidate will default on their loan or not.

Variables

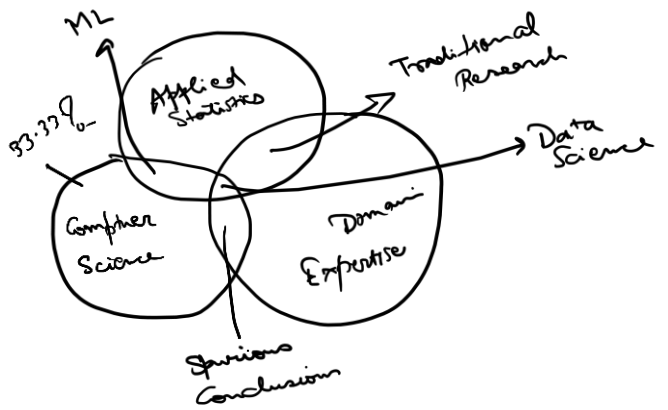
Income	# of cars	Education Level	Loan Amt.	- -	Residency Type	Default (Y/N)
→ 50L	2	M.Tech			3BHK	N
→ —		—			—	N
→						N
.						Y ←
.						Y
.						N
						Y
→ 70L	L	B.Tech	1cr		2BHK	$\boxed{}$ → N

Independent Variables

Training Data

Target Variable.

Income ∝ Default → choosing the variables is called Domain Knowledge.



} Data Science

$y = f(x)$ Eg: $\boxed{y = x^2 + 3x + 5}$

$y = f(x)$

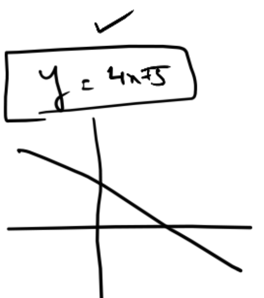
Input (x) → Program → Output (y)

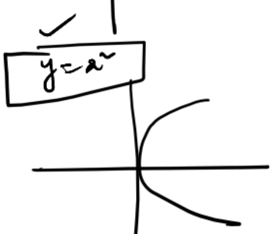
— Input (x) → Machine Learning → Model of the program
— Output (Y) →

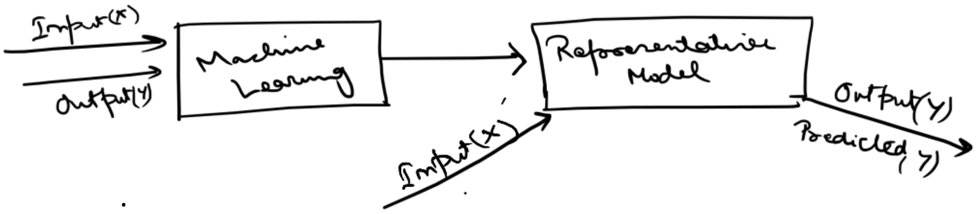
Understand $x \sim y$

↓

Model used on new X to get new Y







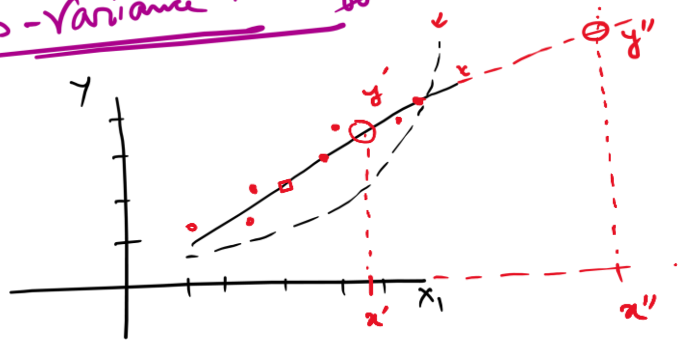
Machine Learning Types

- Supervised Learning
  - Target Variable Discrete → Classification
  - Target Variable Continuous → Regression
- Unsupervised Learning
  - Unlabelled Data
    - Clustering
    - Association
- Semi-supervised Learning
  - Labelled + Unlabelled
    - Classification
    - Clustering
- Reinforcement Learning
  - Positive Reward
  - Negative Reward

(Bias-Variance Tradeoff)

Regression

Stock (Value) X_1	(Demand) X_2	Y (Price)
x_{11}	x_{21}	Y_1
x_{12}	x_{22}	Y_2
x_{13}	x_{23}	Y_3
⋮	⋮	⋮



→ Linear Regression

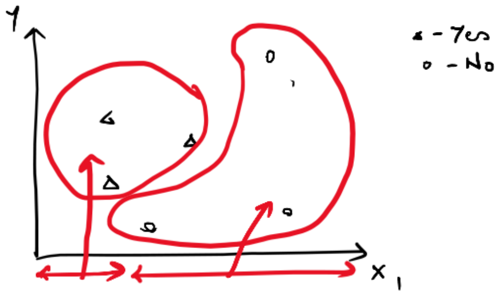
→ Logistic Regression

Is the target variable numerical / categorical?

Classification

(# cars) X_1	(Income) X_2	(Defaulter) Y
3	58L	0
2	28L	1
0	13L	0
1	10L	0
⋮	⋮	⋮

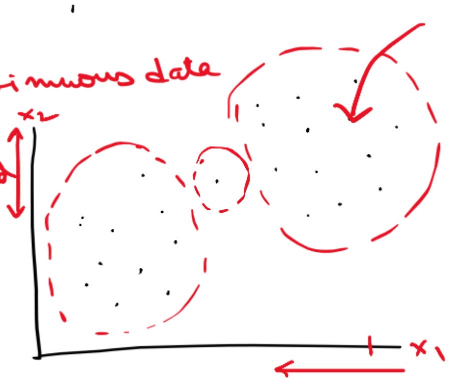
Yes = 1
No = 0



Clustering → classification of continuous data

Numerical
Not labelled

X_1 (# of cars)	X_2 (Income)
⋮	⋮



→ Customer Segmentation

→ Building marketing Strategies

Data → Numerical → Continuous
→ Discrete (Age in completed yrs) [1, 2, 3, 4 ...]

Data → Categorical

Eg:- Good OK Bad

Categorical → Nominal, Ordinal, Binary

Nominal
→ Types of a disease
→ Gender
→ Colour
RGB

Multi class classification.

Ordinal
Pants
L
High
Med
Low

Binary
0, 1
Y, N
T, F

18, 19, 19, 20, 21, 22, 22, 22, 23, 24
→ 18, 19, 19, 20, 21, 22, 22, 22, 22
18, 19, 19, 20, 21, 240, 240, 240, 240

Age
18
20
22
243 →

21 ←

$\bar{x} = 75.75$

Descriptive Statistics

→ Measure of Central Tendency
→ Mean
Median →
Mode

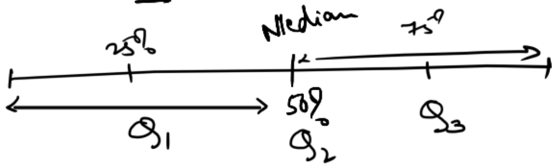
→ Measure of dispersion/variance

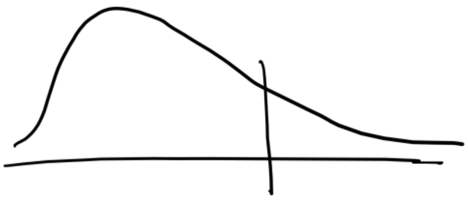
$$\text{Variance} = \frac{1}{n}\sum(x_i - \bar{x})^2$$

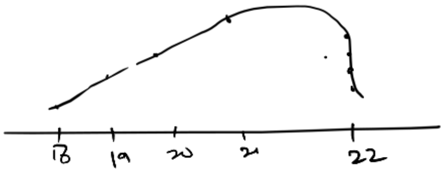
$$IQR = Q_3 - Q_1$$

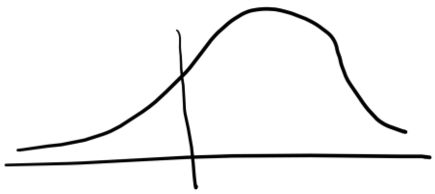
$$\text{Range} = \text{Max} - \text{Min}$$

Skewness









→ Enumerate - index
- value

array - homogeneous
series - {key1: [,], key2: [ ]}

→ Outliers - If < 6% → delete and use the rest
> 6% → investigate

Outliers → Delete, Replace

Replace → Continuous variables (median), categorical variables (mode)

Outliers are detected in 2 ways :- Box Plot / IQR

I

$$\text{Max} = Q_3 + 1.5\,IQR$$

$$\text{Min} = Q_1 - 1.5\,IQR$$

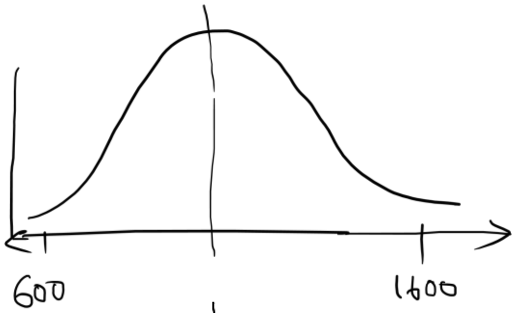
99% of the data should be b/w [Min, Max]

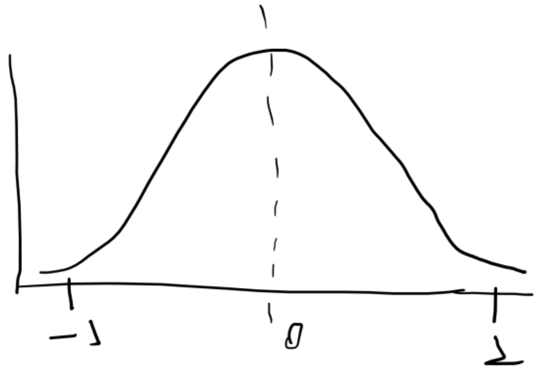
II Z-Score

$$Z\text{-Score} = Z_i = \frac{x_i - \bar{x}}{s.d(x)}$$

$\bar{x} \rightarrow$ Mean

X	Z
x_1	Z_1
x_2	Z_2
x_3	Z_3
x_4	$\vdots$
$\vdots$	$\vdots$
x_n	Z_n





After this transformation, 95% of the data (Z) should lie b/w (-1, 1)

outlier = $Z > 3$ or $Z < -3$

99.5% of the data will be b/w $[Z - 3, Z + 3]$

Z-scoring ⟷ Normalization ⟷ Standardization

Treating categorical variables :- Label Encoding
- One Hot Encoding

Score	Score (LE)	Score Low	Score Med	Score High
Low	0	1	0	0
Low	0	1	0	0
Med	1	0	1	0
High	2	0	0	1
Med	1	0	1	0

Disadvantage (LE) – Mis interpretation of ordinal values
– Arbitrary numerical system
– Complex if too many categories

Disadvantage (OHE) – Too many new variables created

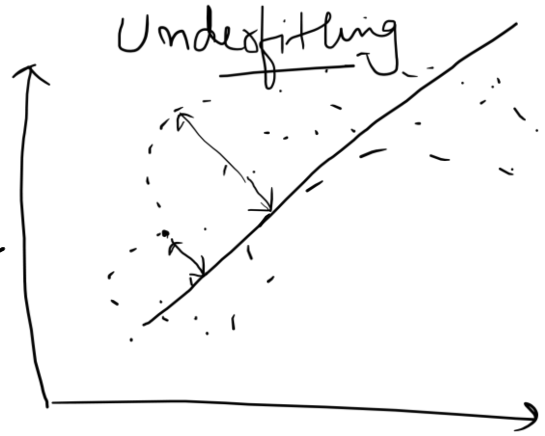
$y = \beta_0 + \beta_1 x_1$

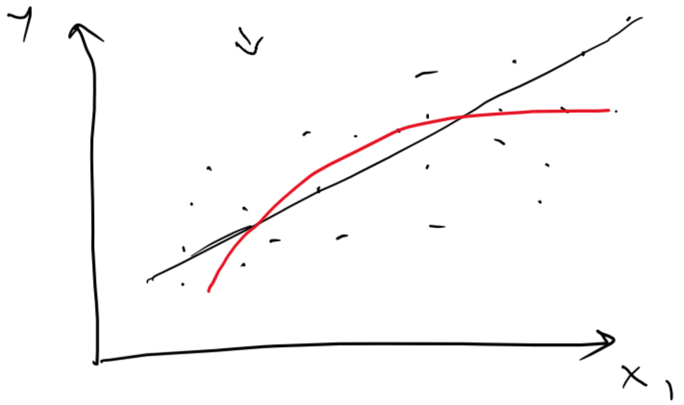
Overfitting

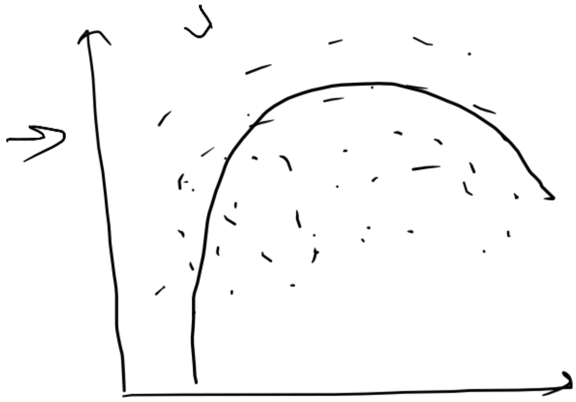
Underfitting

Bias

Var







Bias-Variance Tradeoff

Linear Regression → Linear relationship b/w two variables

→ (It predicts the cont. data in target variable

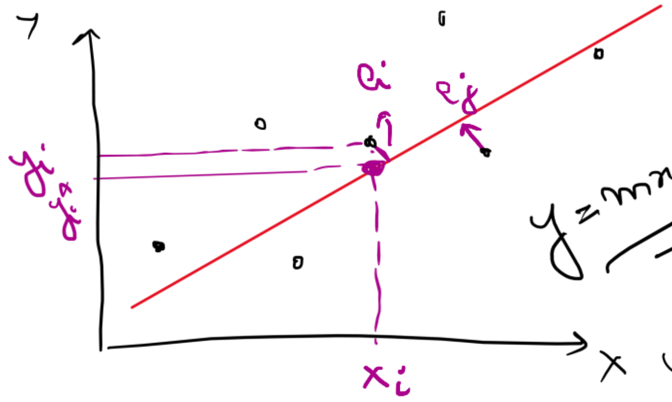
Recap Prior Domain Knowledge → Data Processing & cleaning → Data Splitting → Data Modelling → Data Evaluations

Data Splitting: 80% / 20%

Actual Y	Predicted Y
y_1	$\hat{y}_1$
y_2	$\hat{y}_2$
y_3	$\hat{y}_3$
⋮	⋮

$(y_i - \hat{y}_i)$

X	Y(target)	residual
x_1	y	$y_1 - \hat{y}_1$
x_2	y_2	$y_2 - \hat{y}_2$
x_3	y_3	$y_3 - \hat{y}_3$
⋮	⋮	



$y = mx + c$ → Dist. of the best regression line from the actual target variable.

$y = \beta_0 + \beta_1 x$ → $\hat{y} = \beta_0 + \beta_1 \hat{x}$

Target → Minimize the sum of square of the residuals.

$\sum_{i=1}^{n} e_i^2$ is to be minimized

$$\frac{\partial}{\partial \beta} \sum e_i^2 \Big|_{(\hat{\beta}_1, \hat{\beta}_2)} = 0$$

$$(y_i - \hat{y}_i)^2 \Rightarrow (\beta_0 + \beta_1 x_{1i} - \beta_0 - \beta_1 \hat{x}_{1i})^2$$

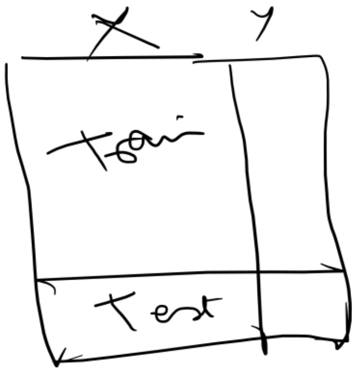
$$y_i = \beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i}$$
$$\hat{y}_i = \beta_0 + \beta_1 \hat{x}_{1i} + \beta_2 \hat{x}_{2i}$$

$$\beta_1 (x_{1i} - \hat{x}_{1i}) + \beta_2 (x_{2i} - \hat{x}_{2i})$$

Best SLR model Line = $\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x$

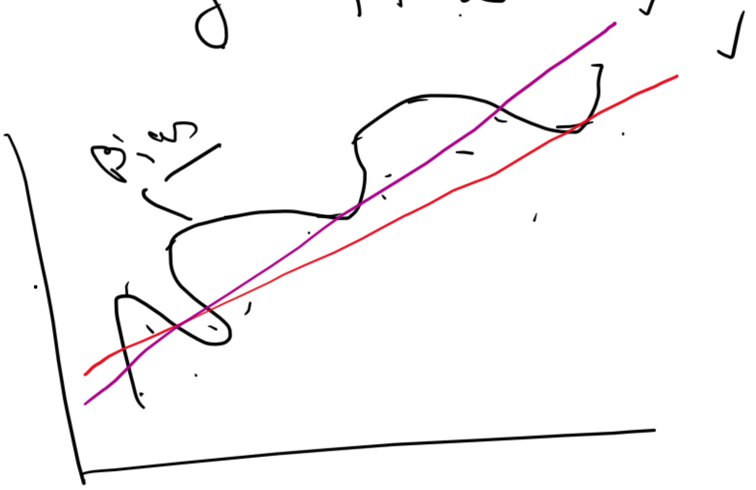
Bias	Var
Errors not explained by trained model → Error in training	Error not explained by test data → error while pred.

$$\frac{\partial}{\partial \beta}\sum\left(y_i - \hat{\beta}_0 - \hat{\beta}_1 x\right)^2 = 0$$



$$\hat{\beta}_1 = \frac{\left(\sum xy - \frac{\sum x \sum y}{n}\right)}{\left(\sum x^2 - \frac{(\sum x)^2}{n}\right)} \qquad \frac{Cov(x, y)}{Var(x)}$$

$$\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x} \qquad e_i = y_i - \hat{y}_i$$



Ordinary Least Square

Bias ↓ → Var ↑

Var ↓ → Bias ↑

SLR → fitted using OLS

⇓

minimizes RSS

Basic Assumption of a SLR model:

- → y_i (target) → continuous
- → (x_1, x_2) → Pairwise independent.
- → ε_i → Homoskedastic — variance should be constant

$$y = \beta_0 + \beta_1 x + \varepsilon_i$$

- → (x_i, y_i) → should have a linear relationship

Recap → 1. What is SLR?
2. Assumptions of SLR?
3. fitting SLR using OLS method
4. Prediction using fitted SLR model

$y = \beta_0 + \beta_1 x \leftarrow$ SLR

→ Predict the target variable using multiple predictors.

$\underset{\sim}{y} = \beta_0 + \beta_1 \underset{\sim}{x_1} + \beta_2 \underset{\sim}{x_2} + \dots + \beta_p x_p \leftarrow$ MLR model

Linear Algebra → n obs. → p indpt. var.

$$\begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_n \end{bmatrix}_{n \times 1} = \begin{bmatrix} \beta_0 + \beta_1 x_{11} + \beta_2 x_{21} + \dots + \beta_p x_{p1} \\ \beta_0 + \beta_1 x_{12} + \beta_2 x_{22} + \dots + \beta_p x_{p2} \\ \vdots \\ \beta_0 + \beta_1 x_{1n} + \beta_2 x_{2n} + \dots + \beta_p x_{pn} \end{bmatrix}$$

$$\underset{\sim}{y} = \begin{array}{c} \begin{array}{ccccc} & X_1 & X_2 & & X_p \end{array} \\ \begin{bmatrix} 1 & x_{11} & x_{21} & \dots & x_{p1} \\ 1 & x_{12} & x_{22} & \dots & \vdots \\ 1 & \vdots & \vdots & & \vdots \\ 1 & x_{1n} & x_{2n} & \dots & x_{pn} \end{bmatrix} \end{array} \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \\ \vdots \\ \beta_p \end{bmatrix}$$

$\underset{\sim}{y} = X \underset{\sim}{\beta} + \underset{\sim}{\varepsilon} \leftarrow$ residual/ Bias/ error

To fit MLR model :- Gauss-Markov Theorem

→ To minimize Residual Sum of squares (RSS)

$$RSS = \sum_{i=1}^{n} \varepsilon_i^2 \qquad [\varepsilon_1\ \varepsilon_2\ \varepsilon_3\ \dots\ \varepsilon_n] \begin{bmatrix} \varepsilon_1 \\ \varepsilon_2 \\ \vdots \\ \varepsilon_n \end{bmatrix}$$

$$= \underset{\sim}{\varepsilon}' \underset{\sim}{\varepsilon}$$

$$= \frac{\partial}{\partial \underset{\sim}{\beta}} \underset{\sim}{\varepsilon}' \underset{\sim}{\varepsilon} \Bigg|_{\underset{\sim}{\beta} = \hat{\underset{\sim}{\beta}}} = 0$$

$$\Rightarrow \hat{\underset{\sim}{\beta}} = (X'X)^{-1} X' \underset{\sim}{y}$$ ← Generalized Least Square Method (GLS)

Assumptions

1. Target variable ($\underset{\sim}{y}$) must be normally distributed
2. (X_i, X_j) must be pairwise independent and there must not be any multicollinearity
3. All ε_i must be independent of each other and have the same variance (spread) with mean 0

   homoskedastic

4. $\underset{\sim}{\varepsilon} \sim N(0, \Sigma)$

$$\Sigma = \begin{bmatrix} \sigma & 0 & 0 & 0 & \cdots & 0 \\ 0 & \sigma & 0 & 0 & \cdots & 0 \\ \vdots & & & & & \\ 0 & \cdots & & & \cdots & \sigma \end{bmatrix}$$

Effects of multicollinearity

— $x_1, x_2, \ldots x_p \rightarrow x_3, x_4$

Variance inflation → cause extra bias towards the impact of x_3 & x_4

← { Impact of other imp variables will get masked/reduced

Preprocessing

Solution : VIF → Variance Inflation Factor

dropped ← x_3 → VIF → > 5 → Drop the variable with highest VIF score

x_4 , x_5 ....

3.7 → 2.7, 1.2 → 0.56

→ We have (p-1) variables

MLR evaluation → SLR → R^2, MAE, MSE, Adj R^2

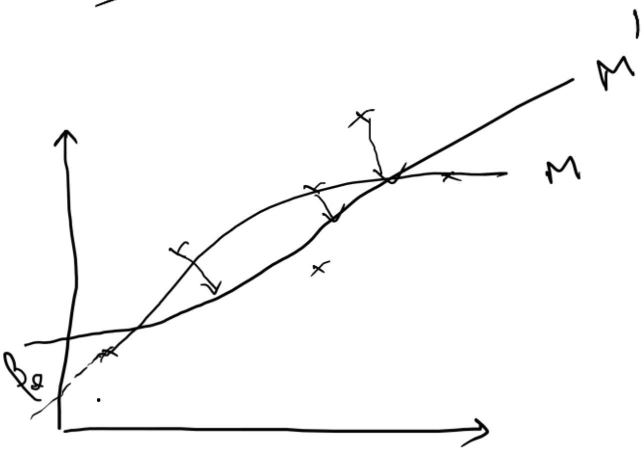
$$\text{Adj } R^2 = 1 - (1 - R^2)\frac{n-1}{n-p-1}$$

→ Why adj R^2 instead of just R^2 ?

p↑ ; normal R^2↑.

H.W :→ 5 ways to reduce underfitting in MLR?

→ 5 ways to reduce overfitting in MLR?



M' → Penalty to M

↓

new RSS is minimized

Penalty → over the model parameters

Ridge Regression

$$RSS' = RSS + \underbrace{\lambda \sum_{i=1}^{p} \beta_i^2}_{\text{Penalty multiplier}}$$

Lasso Regression

$$RSS'' = RSS + \underbrace{\lambda \sum_{i=1}^{p} |\beta_i|}_{\text{Penalty multiplier}}$$

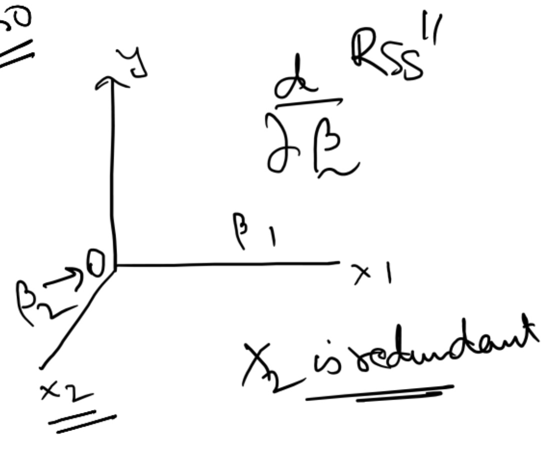
$\lambda \in (0, \infty)$

Diff b/w Ridge & Lasso Regression :-

$\sum \beta_i^2 \to \beta_1^2$ of X_1 → impact will be magnified and it will use all x_i's as important

Ridge

Lasso



If $\lambda < 1$

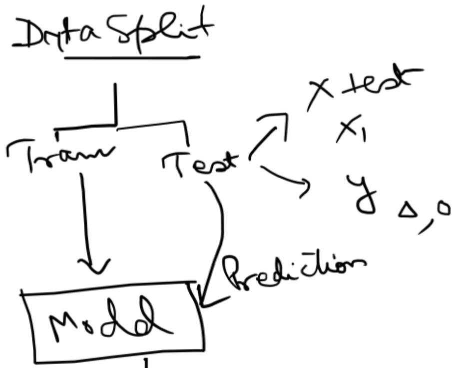
$|\beta_i| \to 0$, help in identifying redundant Variable as $\beta_i \to 0$

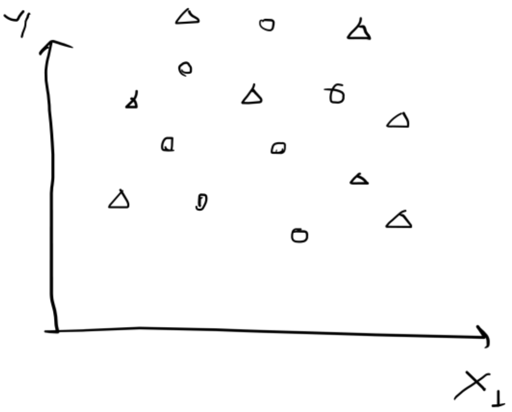
find λ where adj R^2 is max.

Q) Which is better Ridge or Lasso? How to choose b/w Ridge & Lasso? ← H.W.

Classification Day 7

Confusion Matrix :





Predicted class compared with actual test class

y_pred	y test (actual)	
△	△ ✓	
△	△ ✓	
○	△	← miss classification
○	○ ✓	
△	○	← mis
○	○ ✓	
△	△ ✓	
○	△	← mis
○	○ ✓	
△	△ ✓	

n obs.

Annotations

△ - Positive

○ - Negative

Actual

Predicted	△	○
△	True Positive TP (4)	False Positive (1) FP
○	False Negative (2) FN	True Negative (3) TN

How to evaluate a trained Class Model?

① **Accuracy** $= \dfrac{TP + TN}{(TP + FP + FN + TN)} = \dfrac{7}{10} = 70\%$

Detection of cancer in 100 patients (testing set)

Cancer — 1 (+), T

No cancer — 0 (−), F

	Actual 1	Actual 0	
Pred 1	3	2	= 5
Pred 0	5	90	
	= 8		

$$\frac{\text{total patients predicted to have cancer}}{\text{total predicted +ve}}$$

$$\frac{\text{total no. of patients with cancer}}{\text{Total actual +ve}}$$

→ FN is much more dangerous than FP

→ Accuracy = 93%

Actual :- 1 = 8, 0 = 92 } Imbalanced Data

Other more important metrics to evaluate class models:-

(i) Precision

$$\frac{TP}{TP+FP} = \frac{3}{5} = 60\%$$

$$= \frac{\#\text{ correct +ve pred}}{\text{Total +ve pred.}}$$

(ii) Recall

$$\frac{TP}{TP+FN} = \frac{3}{8} = 37.5\%$$

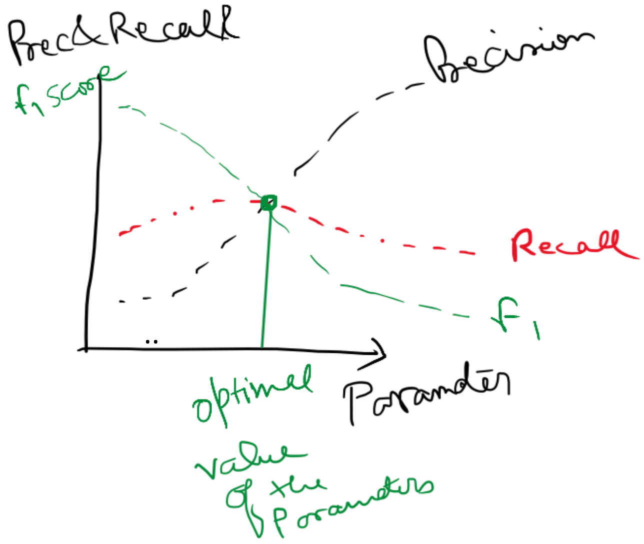
$$= \frac{\#\text{ correct +ve pred}}{\text{Total no. of +ve cases}}$$

→ Higher the values of Precision & Recall, the better the model is.

F_1-Score :- Harmonic Mean of Precision & Recall

$$= \frac{2\ \text{Prec} \times \text{Recall}}{\text{Prec} + \text{Recall}}$$

All classification models will have an iterable parameter over which Prec & Recall will vary



take the value of the parameters for which F1 score is the highest

HW What is an AUC-ROC curve? Why is it important?

$$\underline{FPR} = \frac{FP}{FP + TN}$$

$$\underline{FNR} = \frac{FN}{FN + TP}$$

Overfitting

→ Increasing training size
→ Simplify polynomial order
→ Ridge & Lasso (spl. case)
→ Early stopping *
→ Cross validation

Underfitting

– reduction of noise
– increasing no. of features
– increase polynomial order
– cross-validation
– adjusting hyperparameters

→ HW. What is the difference b/w parameters & hyperparameters?

		Actual Y	Actual N
Pred	Y	TP	$\boxed{FP}$
	N	$\boxed{FN}$	TN

Will loan be recovered?
Y
N

FP is worse

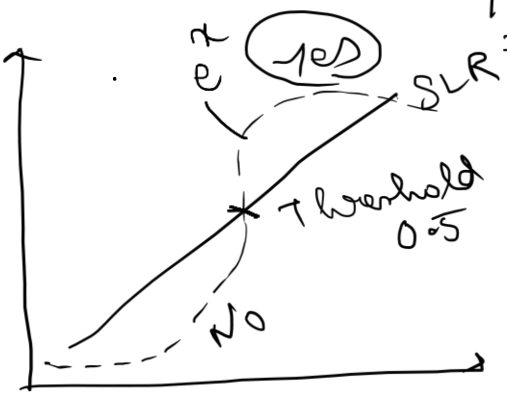
Logistic Regression

X   y Rent an e-bike or not?

Age	Rent
30	No
49	Yes
45	Yes
28	No
47	Yes
32	No
34	No
38	Yes
42	No
55	Yes

$$y_i = \beta_0 + \beta_1 x_i + e_i$$ ← e_i needs to be reduced.

SLR cannot partition b/w target variables



$$\pi = P(Y|X = x_i)$$ = Cond. Prob. of Y when $X = x_i$

$$= f(\beta_0 + \beta_1 x)$$

$$= \frac{e^{\beta_0 + \beta_1 x}}{1 + e^{\beta_0 + \beta_1 x}}$$

$$\frac{\pi}{1-\pi} = e^{\beta_0 + \beta_1 x}$$ → LHS - log odds Ratio

$$\log\left(\frac{\pi}{1-\pi}\right) = \beta_0 + \beta_1 X$$ ← Logit Model

Estimated β_0 & β_1 :- Maximum Likelihood Estimation (MLE)

AIC - Akaike Information Criterion

→ Model selection technique

① Goodness of fit
↓
How well the model explains the data

② Model complexity
↓
parameters

$$AIC = 2k - 2\ln(L)$$

k → # Parameters

L → Max likelihood func of the model

Lower AIC
⇓
Better Model

→ Not measure the accuracy of the model

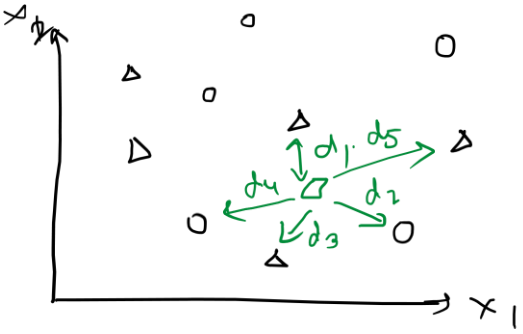
→ Penalize the more complex models.

KNN

10 obs

X_1 X_2 Y

o △



K = No. of NN we want to consider.

$\boxed{k=4}$

□ ∈ test data

Dist		Train
d_1	△	△
d_2	*	o
d_3	△	△
d_4	o	o
d_5	*	△

Majority

△ = 3

o = 2

Tre

So the predicted dot is △.

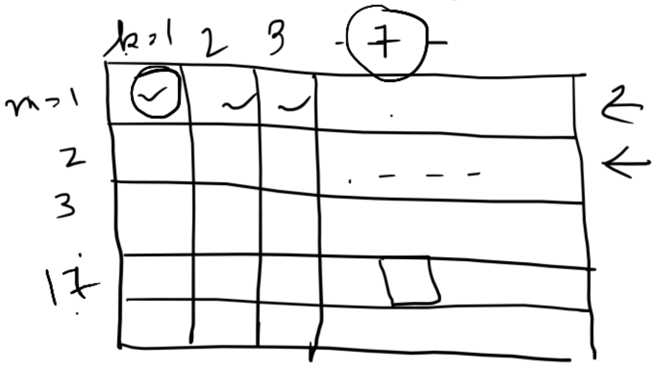
How to choose k?

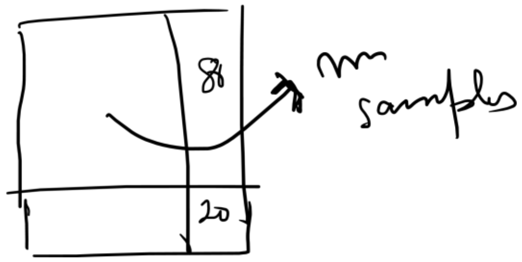
① k should be an odd no. [only when it is a binary classification]

② Start with $k=3$ and iterate till $k=\sqrt{n}$

Eg $n=400$ $k=[3,4,5,\ldots 20]$ $k_{opt}=k\big|_{max+10}$

③ Cross Validation – Using diff subsamples of the training data



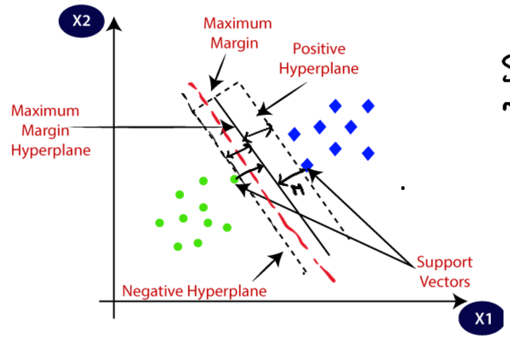


each cell → the performance metric of the value of k.

SVM Support Vector Machine — Eager Learner

Parameters — Internal variables — Estimated (Training Data)
(coeff. in LR)
(weights in DT, Log. Reg)
— Automatically updated — Minimize the loss func. using Grad. descent (OLS)
Slope of the tangent at any particular point in the curve (Can also be named for Grad desc)

Hyperparameters — External variables → Pre-set → Control the training process and the arch. const. of the model
(Regularizations L1, L2)
(Kernel in SVM)
(No. of branches in DT)
→ Not Estimated rather set manually or by automated tuning.

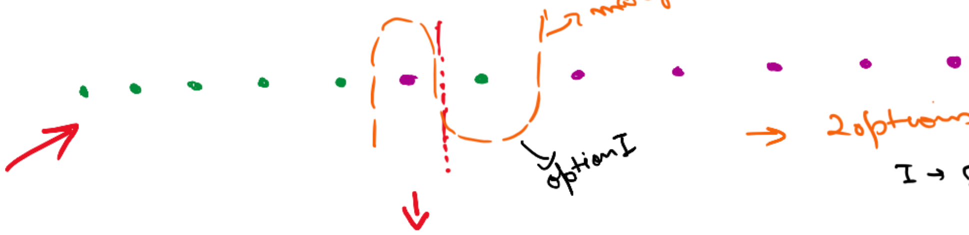


SVM will try to fit a hyperplane that separates 2 class as much as possible.
Maximum margin hyperplane is fitted such that M on either side is equal.
M → margin

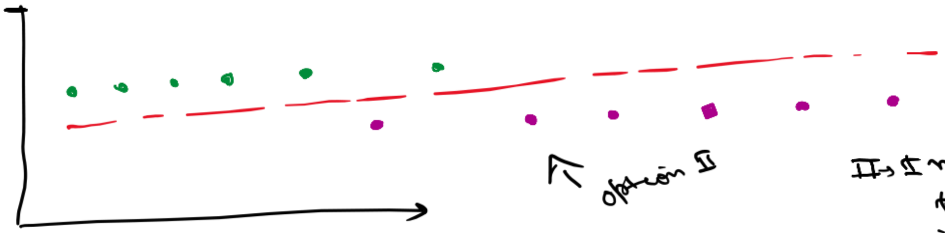
Principal : Higher the value of m, better is the classification

Eg 2

Eg 3

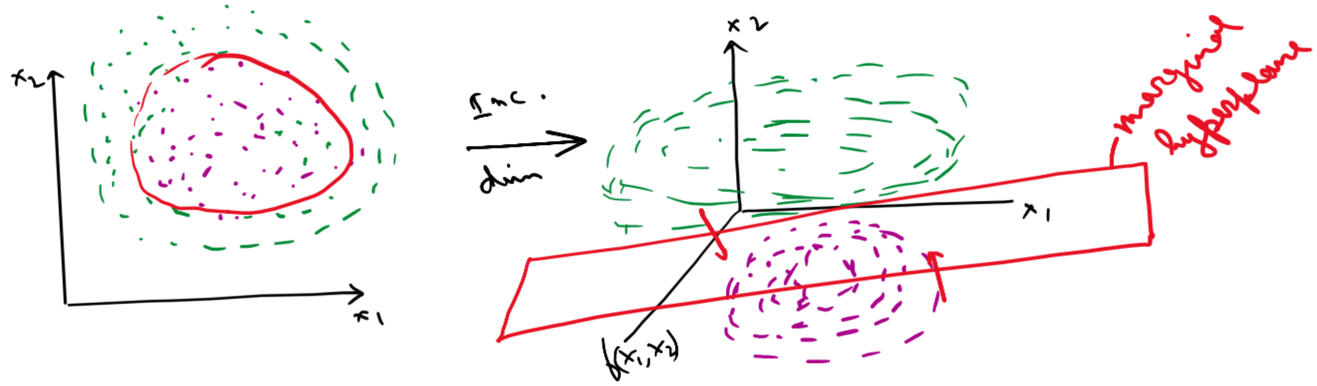


→ 2 options of classification
I → Introduce bias to decrease var., reduce misclassification, increase precision



II → Introduce a new dimension or plane as the function of the independent variables

Eg 4



→ Black box model
→ Cannot retrace model path
→ SVM is very resource intensive

Understanding the parameters of SVM

① Kernel type → SVM can perform iterative fitting based on kernel type

Kernel types → Linear
→ radial function - rbf
→ polynomial →
→ 'precomputed' function

② Penalty parameter → $c > 0$
↳ Lower the value of c, lesser is the introduced bias

③ Gamma Regularization → $g \in \left(0, \frac{1}{n}\right)$
↳ n is the no. of obs. in training data
↓
iterating over the entire range.

low ↓ less surrounding data pts will be considered to find the max. margin hyperplane.

high ↓ takes larger surrounding data pts.

→ SVM's are computationally expensive
↳ Dynamic & parallel computing
↳ Increase your resource

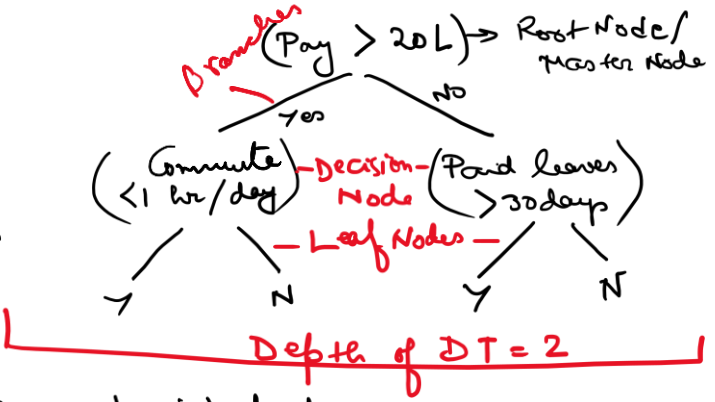
→ Big data → Partition clusters
3PB
↳ samples (150 TB) each.

C_1 C_2 C_3 parallel
aggregated
↓
output

In SVM it is impossible to retrace the path chosen by the model to fit the max. margin hyperplane.
So we cannot interpret the model parameters.

Decision Trees

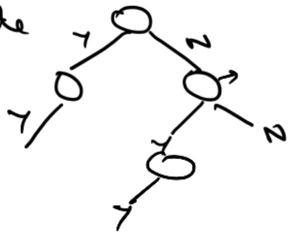
→ Whether to accept a job offer or not?
→ Pay > 20 Lakh
→ Commute to work < 1 hr per day
→ Do they give paid leaves > 30 days



Q) Key advantage of using DT? → Very easy to interpret

→ Factors that determine the no. of partitions the algorithm will make?

① Root Node



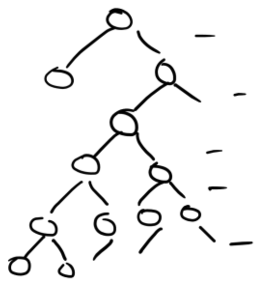
② X_1, X_2, X_3, X_4

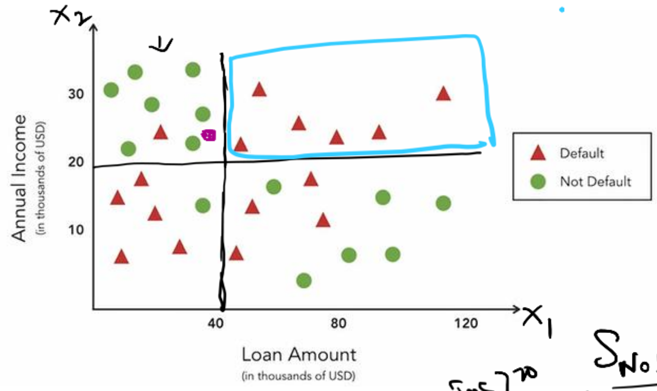
$(X_1, X_2), (X_1, X_2, X_3)$

(X_3, X_4) (X_2, X_3, X_4)

All combinations are exhausted
↓
Partitioning will continue

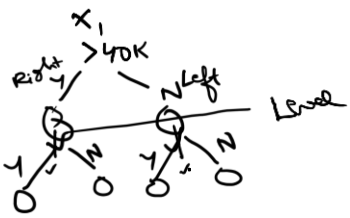
③ Specified Constraints
max_depth = 5





Total Data pts = 30
$\Delta = 16$
$O = 14$

Steps:
① Create a partition using any one variable so that Impurity on either side of the partition is least



Annual Inc > 70

$S_{\text{No partition}}$
$$2\begin{cases} \Delta = \frac{16}{30} \\ O = \frac{14}{30} \end{cases}$$

After Partition (S_1)
$$2\{S_{left} = \Delta = \frac{6}{14}, \quad O = \frac{8}{14}$$
$$2\{S_{Right} = \Delta = \frac{10}{16}, \quad O = \frac{6}{16}$$

After Partition (S_2)
$$\boxed{S_{UL} = \Delta = 1/8, \quad O = 7/8}$$
$$S_{DL} = \Delta = \frac{5}{6}, \quad O = 1/6$$

Q) Has Impurity dropped after partition? How is Impurity calculated?

$$\rightarrow \text{Entropy}(S) = \sum_{i=1}^{c} p_i \log(p_i)$$

C = No. of unique classes
p_i = proportion of classes in each side

I $\text{Entropy}_{NP} = 0.997$

Left = 0.47
Right = 0.53

$(X_1 = 40 \text{ in } S_1)$

II Any partition S_1 has been considered

$$\left.\begin{array}{l} \text{Entropy}(S_{left}) = 0.986 \\ \text{Entropy}(S_{right}) = 0.961 \end{array}\right\} \text{Net Entropy}(S_1) = \sum_{i=1}^{n} w_i \, En \, S_i = \underline{0.967}$$

n = No. of partitions

III $\text{Info Gain}(S) = \text{Entropy}(S_1) - \text{Entropy}(\text{No partition})$
$= 0.997 - 0.967 = 0.03$

Partition is selected in such a way, so that Info Gain (S) is maximum.

Same II iterative steps will be conducted for next variables.

↳ Info Gain due to S_2 must be max.

How to classify a new observation?

X_1	X_2	Y
35	24	?

Which Node to select?

$X_1 < 40$ & $X_2 > 20$

□ ⟶ O $\frac{7}{8}$ △ $\frac{1}{8}$

So the new obs. is classified to the category with highest prop.

For DT Regression:-

$(X_1 < 40$ & $X_2 < 20)$

$\underbrace{(4.5, 5.7, 7.8, 6.7, 3.4, 9.1)}_{\text{agg avg}}$

X_1	X_2	Continuous Data
30	15	9.1

Overfitting in Decision Trees

X_1 X_2 X_3 . . . X_p $\frac{Y}{A,B}$

1
2
3
:
n

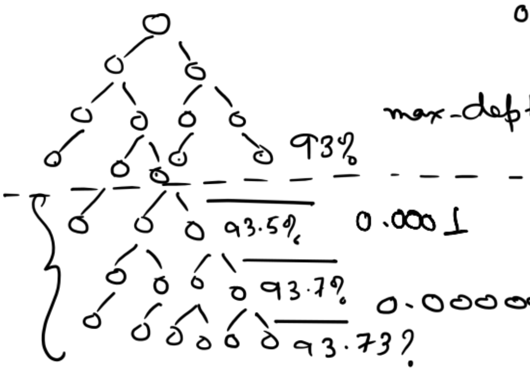
n - data pts.
p - indpt var.

→ Prone to overfitting when $n < p$

→ Soln ① Increase 'n'

(II) Use max_depth to restrict over complication

Rule of Thumb: Use DT only when $n > 30$



max_depth = 3

0.0001

0.00000001

x	y
train	80%
test	20%

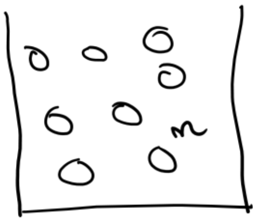
Bagging: Take sub samples of X-train to create multiple DT ⟶ agg the decision

with replacement sampling

train 1 $\xrightarrow{\text{predict}}$ y pred 1 ⟶ Perf 1.
train 2 ⟶ y pred 2 ⟶ Perf 2
" 3 ⟶ " 3 ⟶ Perf 3
" 4 ⟶ " 4 ⟶ Perf 4

agg. final output

Bagging in a DT reduces both underfitting & overfitting as much as possible.



n balls in the bag are numbered as

1 2 3 4 . . . . . . n

I → Create a sample of 20 balls → Pull out 20 balls at random from the bag.

↳ (n − 20)

→ Create a 2nd sample of 20 balls from whatever is left

↳ 20 balls is diff from the 1st sample

↳ (n − 40)

Without replacement

II Create a sample of 20 balls → I check them → Put them back in the bag

1 2 4 8 . . . n

With replacement

Create a 2nd sample of 20 balls → Some of the numbers will repeat from the 1st sample → Put them back

4 8 10 13 . . n

Ensemble Techniques

→ All these branches or sample tray will have attached bias in case of any seasonal or underlying trend

WOR

train → 25K 25K 25K 25K

mutually exclusive

Eg:- 1 Lac data pts

↓

Structured

↓

Does not have any underlying trend

X_1 X_2 X_3 ... X_p	Y
.	A
.	B
.	
n = 1 L	
→	

Let's consider a new obs. G.

use W.R.

	1	2	3	4
	75%	66%	80%	90%
Train	↓ DT_1	↓ DT_2	↓ DT_3	↓ DT_4
Test	↓ pred 1	↓ 2	↓ 3	↓ 4
	↓ A	↓ A	↓ B	↓ A

majority vote

New obs G → A

Bootstrapping ↓ random sub sampling from a data to perform any opr.

Bagging → Bootstrapping → Bootstrapped models → Aggregating

Q) If an ML model is not affected by missing values/outliers, do we need to remove or treat them before modelling?

→ No.

→ only DT is not affected by MV/outliers. but other models are.

→ So treatment/removal of MV/outliers is compulsory

Bagging should be done with same classification model fitted on all subsamples. → Different hyperparameters for different subsamples may be used

train → ① KNN
train → ② SVM
train → ③ DT
train → ④ Log.Reg

aggregation tw we do will not be homogeneous.

what will be the final model be?

Q) What happens if each sub sample will also have a sample of indpt var?

→ Random Forest

$X_1 X_2 X_3 \dots X_p$ — train | Y: A, B, A, …

n obs.

train:

Sample ① $n_1 = 75\%$ of n → DT_1 → Pred ① - A
$p_1 = X_1 X_2 X_7 X_{10} X_{15}$

Sample ② $n_2 = 90\%$ of n → DT_2 → Pred ② - B
$P_2 = X_7 X_{10} X_{11} X_{13}$

Sample ③ $n_3 = 40\%$ of n → DT_3 → Pred ③ - A
$P_3 = X_5, X_{13}, X_{19}, X_{20}$

⋮

Sample (k) → DT_k → Pred k

majority vote aggregation

Total 35 samples ← K
A - 15
B - 20

Now observation
⑤ $x_1\ x_2 \dots x_p$ → B

→ In Python hyper parameters will be same by default → In order to have a hyper parameter Const. → specify that.

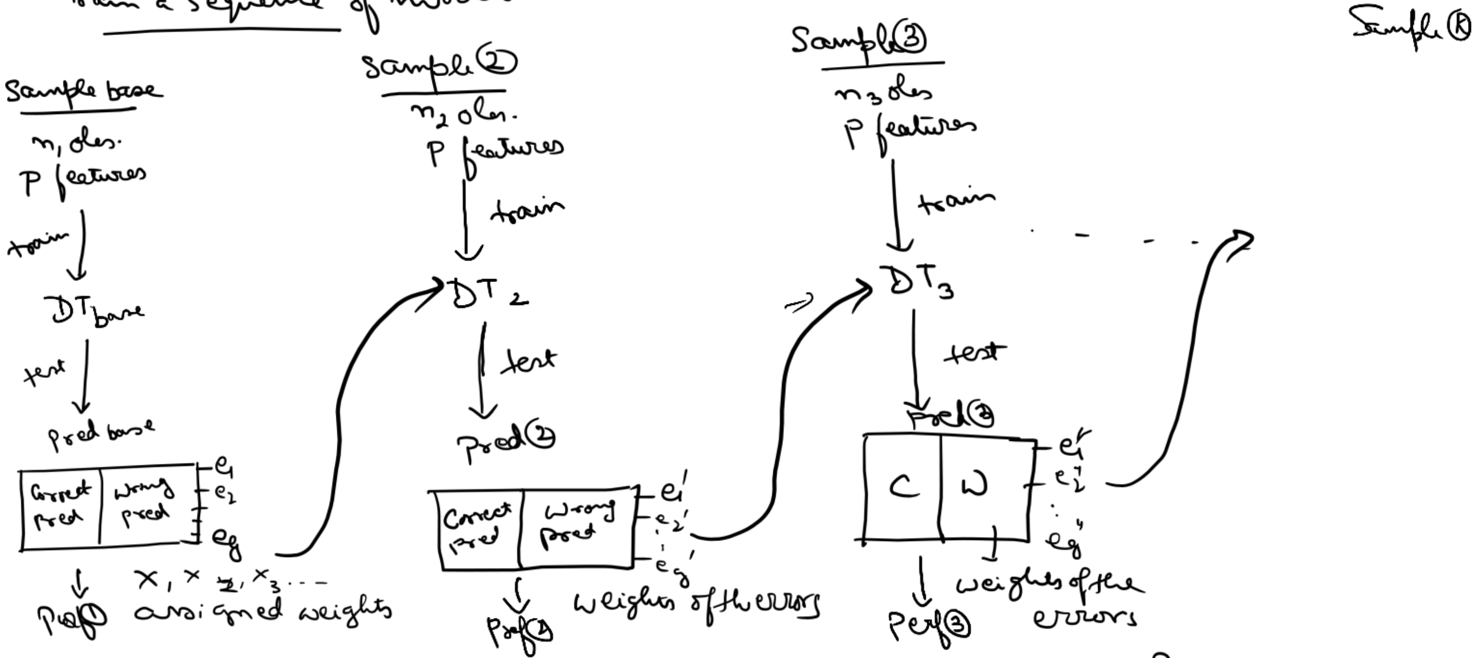
→ Multiple RF fitting across range of hyper parameters rather than using diff. hyper parameter values in same RF

Boosting

Data $x_1 \quad x_2 \quad x_3 \; \ldots \; x_p \qquad y \in (A, B)$

n obs.

train a sequence of models.



How long does it continue / when to stop this iteration of the seq.?

① Performance of the subsequent models have saturated

② Given threshold of the model performance is achieved

Calculation of assigned weights on misclassification of one sub-model in boosting

Naive Bayes

Obs	X_1 (Weather)	Y (Play football or not)
1	Sunny	Yes
2	Sunny	Yes
3	Rainy	No
4	Cloudy	Yes
5	Sunny	No
6	Rainy	Yes
7	Cloudy	No
8	Sunny	No
9	Rainy	Yes
10	Cloudy	Yes
11	Cloudy	? — Yes

Problem to solve :-

* Given the weather condition as 'Cloudy' to predict if the person will play or not

Cond. Prob

$$P(Y=Yes \mid X=Cloudy) = \frac{P(Y \cap X)}{P(X)} = \frac{2/10}{3/10} = \frac{2}{3}$$

Bayes' Theorem

$$P(Y|X) = \frac{P(Y) \cdot P(X|Y)}{P(X)}$$

Cond prob. of target can be represented as the func. of the cond. prob. of the features

$$\underbrace{P(Y|Cloudy)}_{\text{model output}} = \frac{P(Y) \cdot P(X=Cloudy|Y)}{P(Cloudy)}$$

training

Prior prob. → $P(X)$, $P(Y)$ because they can be computed independent of each other.

Posterior prob → $P(X|Y)$ because it calculates the occurence of X based on Y

$P(X=Cloudy|Yes)$

$= \frac{P(X \cap Y)}{P(Yes)}$

$= \frac{2/10}{6/10} = \frac{1}{3}$

$P(X=Cloudy|No)$

$= \frac{P(X \cap Y)}{P(No)} = \frac{1/10}{4/10} = \frac{1}{4}$

$P(Y=Yes \mid X=Cloudy)$

$$= \frac{\cancel{6}/10 \cdot \frac{1}{3}}{\frac{3}{10}} = \frac{2}{3}$$

& $P(Y=No \mid X=Cloudy)$

$$= \frac{4/10 \cdot \frac{1}{4}}{\frac{3}{10}} = \frac{1}{3}$$

Since for X = Cloudy,
$P(Yes|Cloudy) > P(No|Cloudy)$
Y_{pred} = 'Yes'

X_1 (Weather) — Categorical

X_2 (No. of players) — Discrete

X_3 (Age) — Continuous

Y (Play or not) (Yes, No)

$$P(y \mid X_1, X_2, X_3) = \frac{P(y) \cdot P(X_1|y) \cdot P(X_2|y) \cdot P(X_3|y)}{P(X_1)\ P(X_2)\ P(X_3)}$$

Bayes's for p predictors :-

$$P(y \mid X_1, X_2 \dots X_p) = \frac{P(y) \cdot \prod_{i=1}^{p} (P(X_i \mid y))}{\prod_{i=1}^{p} P(X_i)}$$

CLT
Any distⁿ
↓
Normal
as $n \to \infty$

7H 3T
60H 40T
550H 450T
50KH 50KT

Violations in assumptions present :-

① If $X_i \propto X_j$ $i \neq j$ → Drop either X_i or X_j

② Continuous features not normally distributed
→ (i) Increase # observations
(ii) Transform the entire data so that X_i is Normal
(iii) Outlier removal / Treatment to check normality
(iv) Convert the var into robust categories

Age	Age (11-15 A, 16-20 B, 21-25 C, 26-30 D)
18	B
25	C
24	C
13	A
12	A
11	A
19	B
27	D
29	D

Python : 3 algorithmic parameters for NB

I> Bernoulli – for binary / multiclass categorical target

II> Multinomial – for discreate targets / contⁿ targets convertible to categories

III> Gaussian – for continuous targets
* Gaussian NB can be used accurately for any type of target

Eg

Person developed an web-app.
→ Improve the performance
→ Page load time
→ Interactive features in the app
→ No. of components
→ Server Bandwidth
→ Image Compression

– X_1 (Avg # clicks in the page) – Discrete
– X_2 (No. of components) – Discrete
– X_3 (# Interactive features) – Discrete
X_4 (Cache Images present or not) – Categorical
* – X_5 (Cache allocation) – Continuous

Y (Perf.) → Continuous

Which model to fit?

i) Y – Continuous — Regression / clustering / Robust Categories for classification

0 – 10
10 – 20
20 – 30

$P(Y = 20\text{-}30 \mid X_1, X_2 \dots X_5)$
=
?

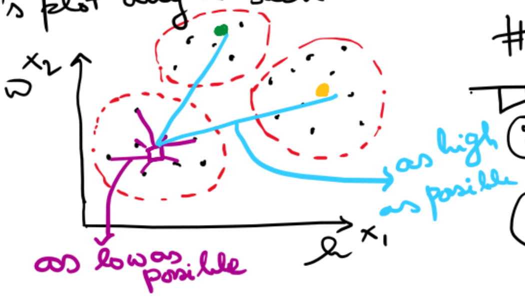
What is K fold Cross-validation?

What is Grid Search / random Search / Bayesian Search ?

Clustering

Eg X_1 (height) X_2 (weight) X_3 (Age) | y (Income/Income group)

Let's plot any 2 such variables :-



Clusters are not alloted based on visual inspection

There are 2 objectives :-

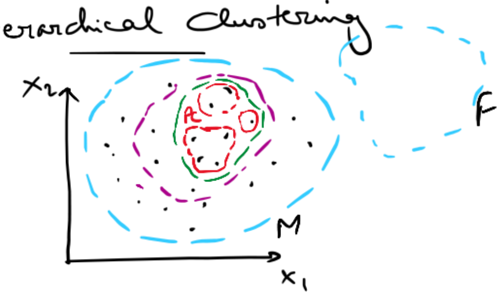
(i) High intra class similarity — within cluster variance very low/as low as possible

(ii) High inter class dissimilarity — Between clusters variance very high

If there are 3 clusters A, B, C

Dist b/w A & (B, C)
B & (A, C)
C & (A, B)
} these are as high as possible

Hierarchical Clustering



C_1 - Male

C_2 - Postgrad Males

C_3 - Income > 50 K with postgrad and Male

C_4 - Paying rent, Income > 50 K, postgrad, Male

Each cluster is created so that within-parent cluster segregation of 2 clusters is as high as possible

Parent		Child
C_1	>	C_2, C_3, C_4
C_2	>	C_3, C_4
C_3	>	C_4

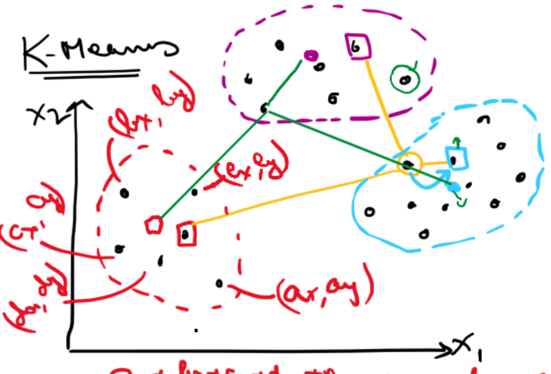
} → Child clusters

Based on the algorithm 2 partitional clustering are :-

i) K-Means Clustering

ii) DBSCAN –

K-Means



$$C_1 = \frac{a_x + b_x + c_x + d_x + e_x}{5}, \left(\frac{a_y + b_y + c_y + d_y + e_y}{5}\right)$$

X_1, X_2 have n obs.

Algo :- Step 1 :- Choose how many clusters we want (K)

Step 2 : Assign 'k' random pts as starting cluster centers

Step 3 : Distance of every obs. is calculated from the current cluster centers

Step 4 : Assign each obs to the cluster of their closest cluster center. So that now we have new clusters

Step 5 :- Calculate new cluster centroids and use them as new cluster centers

Iteration of 3-5 occurs

Q) How to choose k?

→ We will go on iterating (say 3, 4, 5) until clusters are not shifting location
↳ we choose the k clusters obtained as the final clusters.

→ From domain knowledge

→ If k is completely unknown, $k = \sqrt{n/2}$, is a good starting pt. when n is small

$n = 10{,}000$ $\quad k = \sqrt{n/2} \sim 70$ (too expensive).

→ Use 3 different metrics as a judge of optimum 'k' :-

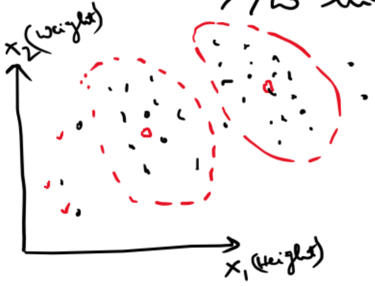
I Elbow method

II Silhoutte score method

III C-H Score

DBSCAN

→ K-Means can only create hyper spherical clusters.
→ does not recognize density based clusters i.e., cannot distinguish b/w high v/s low density group of data.



→ KMeans will not work with spatially dense data

Some Key Concepts :-

*(i) ε → fixed distance from any obs. in the data space

(ii) Neighbourhood → data space around an obs. with radius ε ⇒ $N_p(\varepsilon)$

(iii)* Min-pts → declared no. of obs. present within neighbourhood of a point

(iv) Corepoint → an obs. is a corepoint iff within the neighbourhood of the point, there are atleast 'min-pts' no. of observation.

(v) Borderpoints → " " " " borderpt. iff within the neighbourhood of the point, there are less than 'min-pts' no. of observation.

(vi) Outlier

Eg :- min-pts = 4

* is a hyperparameter in DBSCAN

→ ε - Parameter to be specified in the neighbourhood

→ min-pts - reqd to qualify as corepts.

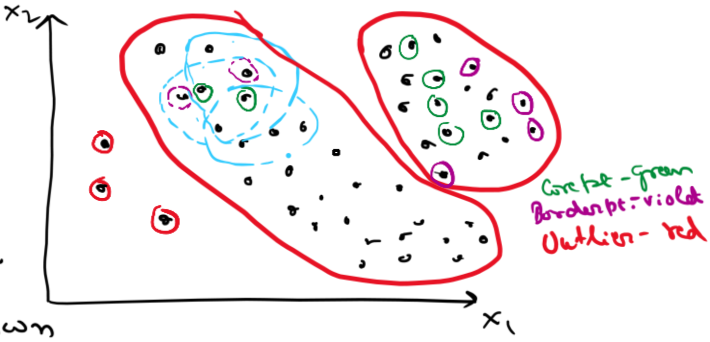
Core pt. $N_A(\varepsilon) \geq$ min pt

→ Borderpt. $N_B(\varepsilon) <$ Min pts

Cluster less outlier

All outliers are also Border points

High density pts are part of a cluster min-pt = 4
v/s
Low density pts are cluster less.



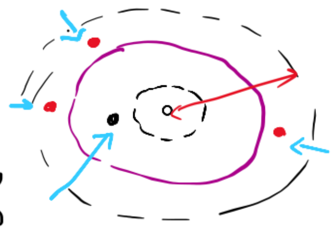
→ Fix ϵ – Min neighbourhood distance
min-pt – minimum no. of data pts in the neighbourhood to qualify as core pt

→ Each and every pt. is considered to create their own neighbourhood, and checked for core pt. or border pt.

Core pts will induct all other pts in their neighbourhood as a part of their cluster
Border pts will try to be inducted as a part of the cluster if there is a core pt., in their neighbourhood, but will not be able to induct any other pt into their cluster.

Outliers are pts that has no other obs. in their neighbourhood.

DBSCAN is also used to detect outliers.
→ DBSCAN doesn't require outlier detection before fitting



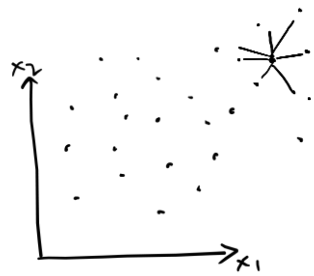
Q) How to choose optimum value of ϵ & min_pts.?

If ϵ is high → too less clusters → underfitting → no detection of outliers

ϵ is low → Small neighbourhoods → overfitting → unnecessary outliers

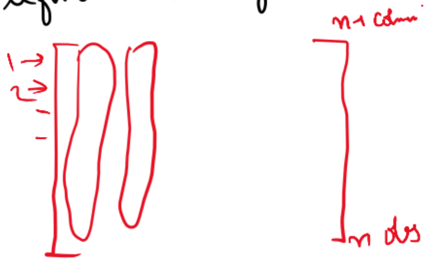
Deciding optimum ϵ :-
(i) After scaling all continuous variables; run iteration over a range of ϵ and also minpts

(ii)

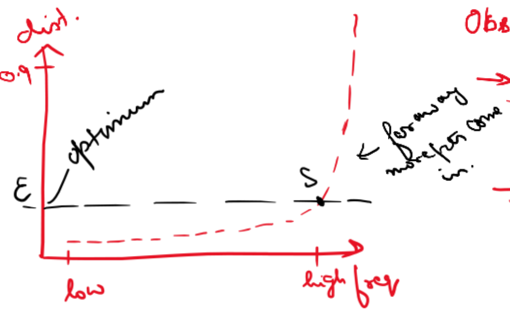


For each obs, we will find the dist. from all other data pts.

Obs 1 : (n-1) dist.
[- - - - -]
Obs 2 : - - - - - -
⋮
Obs n : [ ]



→ 0.02 b/w
→ 0.02 b/w



→ avg. distances need to be obtained
↳ creating a freq table.
→ Sort the distance of every pt. with other pts in the space and plot them.

→ With increase in distance, freq. of other pts increase, i.e., closeness reduces.
→ dist < S → over clustering / overfitting
& dist > S → under clustering / underfitting

Association Rules/Market Basket Analysis → Figure out which items go together.

WALMART, TARGET

← Departmental Stores

← They sell almost all kind of products

* Quantity is not a concern/not considered

Transaction	Items bought
1	{bread, milk, eggs} -
2	{batteries, remote, bread}
3	{milk, bread, chocolate} -
4	{chips, biscuits, milk, egg, tomato} -

Objective: If a person buys Item {A}, how likely are they going to buy Item {B}

① Itemset → Set of unique items in a transaction

② $\text{Support}\{\text{Item}\} = \dfrac{\text{Freq}\{\text{Item}\}}{\text{Total Transactions}}$, For eg:- $\text{Supp}\{\text{milk}\} = \dfrac{3}{4} = 75\%$

③ Confidence{X, Y} = If X is bought, probability of Y being bought together with X.

$$= \frac{\text{Support of }(X, Y)}{\text{Support of } X}$$

$P(A|B)$?

④ $$\text{Lift}\{X, Y\} = \frac{\text{Support}(X, Y)}{\text{Support}(X).\text{Support}(Y)}$$

$\text{Corel}(X, Y) \overset{?}{=} \dfrac{E(X, Y)}{E(X).E(Y)}$

\# Putting context to a relationship b/w 2 items always depends on the analyst's perspective

Rule Set/Association Rule: $\underset{\text{antecedent}}{\{X\}} \longrightarrow \underset{\text{Consequent}}{\overset{\text{then}}{\{Y\}}}$ If X is bought then Y is bought

Eg: {milk} → {eggs} : Z

Scenario where milk is bought so eggs may be bought!

Lift{Z} → Lift is a metric that can be used to evaluate the strength of the association rule. → Zhang's metric can also be used

\# Applications of Association rules:-

→ Healthcare - Drugs and their efficacy against diseases

→ Social Media Posts

→ Stock Analysis

→ Food order

Frequent vs Infrequent Itemsets

A: {bread, milk} → eggs

Supp(A) > threshold

frequent

B: {batteries} → {bread}

Supp(B) < threshold

Infrequent

Q) How to determine threshold?
→ Threshold is the only hyper-parameter for Association Rule algo.

Using this Apriori Method to find Association Rules:
→ We get 3 types of insights :-

Obvious	Absurd	Actionable
rational context and can be easily identified {bread, milk} → {eggs}	No rational context can be easily identified {Diapers} → {beer}	rational context, is developed by domain or in practicality/action → {Sock, shoes} → {umbrella} [Context: It is the rainy season and schools, colleges & offices have started]

Day 16 Dimension Reduction
- Feature selection → Select only non redundant & high var features
- Feature Extraction → Extract a subset of features or a comb. of these to create a new feature which best represents the original set!

→ Feature Engineering :- We have 10 features $x_1, x_2, x_3 \dots x_{10}$ $p = 10$
→ The idea of dimension reduction is to use $k < p$ features but not lose data/information as far as possible.

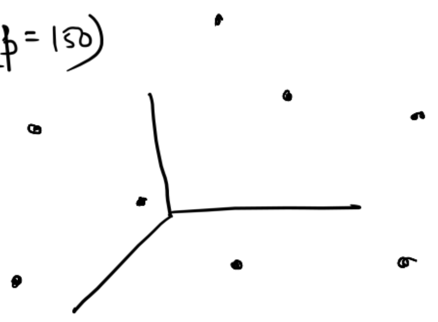
Q) Where in the process flow does this fit in?

Domain Knowledge Business Prob → Data cleaning & Pre-processing → $\boxed{\text{EDA, Visualization}}$ → Data Splitting → Model Building → Pred & Evaluate → Deploy

→ Dimensionality Reduction is performed on the entire data and not just training. Hence it is done on clean & preprocessed data to make it ready for modelling.

Curse of Dimensionality: Data set has p features ($p = 150$)

x_1	x_2	---	x_p
3.1	4.2	....	7.1



Smallest gap ~ largest gap

→ When working with high dim data, data tends to becomes sparse as the volume of the space increases exponentially
→ Distance metrics become meaningless
→ overfitting

Soln : ① Dimensionality Reduction — PCA
② Feature Selection —
③ Regularization : L1 / L2

Decrease model complexity as well as tackle overfitting

Regression: Feature selection methods :-

① Forward Selection :-

[Data: p features & n obs. ; $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 \dots + \beta_p x_p$]

Start with a null model :-

$y = \beta_0 \rightarrow$ null model

p possibilities are there

I $\quad y = \beta_0 + \beta_1 X_1 \rightarrow Adj\ R^2_{I}$

II $\quad y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 \rightarrow Adj\ R^2_{II}$ MSE

$\vdots$

In total there are $p!$ possibilities

$p \quad y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 \dots + \beta_p X_p \rightarrow Adj\ R^2_p$

If $Adj\ R^2$ of any step is greater than prev. step then the added feature in that step is considered 'important'. Else, the added feature is non relevant / redundant ↳ choose to drop it

Backward Selection:

Starting with full model and proceed to the null model

I : $y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 \dots + \beta_p X_p$

II $\quad y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_{p-1} X_{p-1}$

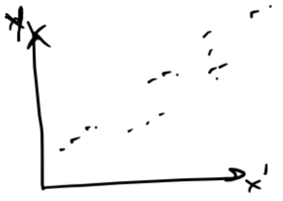
$\vdots$

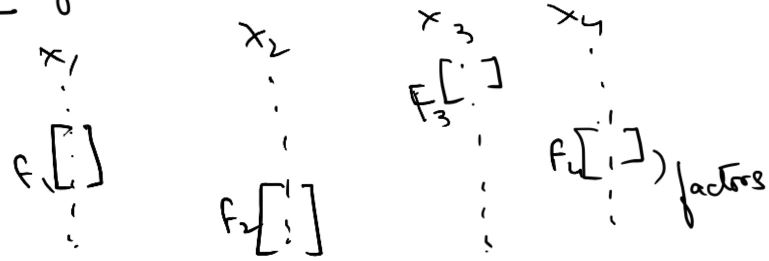
} Compare the perf. using $Adj\ R^2$, MSE

If we see that $Adj\ R^2$ is increasing or remains same, that var. was redundant,
Else, we have to drop some other var and check.

In clustering problem, we can use this on a subset of features to get the feature importance, so that we can exclude redundant variables and use only relevant var for the clustering algo.

Feature Extraction : Factor Analysis



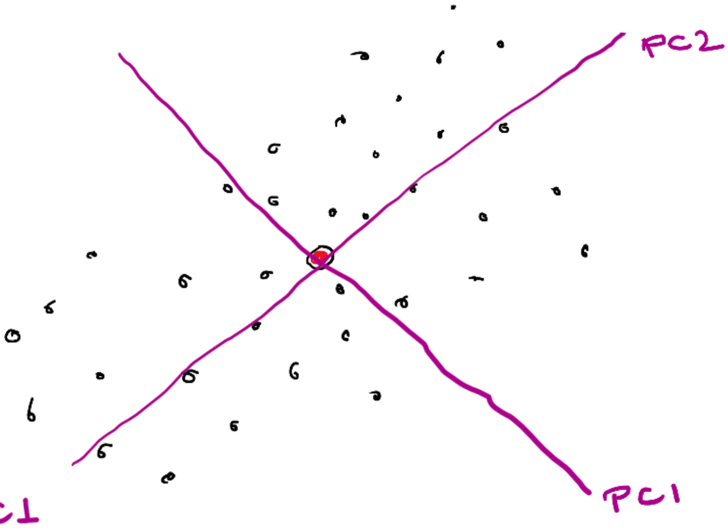


We get a new feature $X' = L_1' f_1 + L_2' F_2 + L_3' F_3 + L_4' F_4$

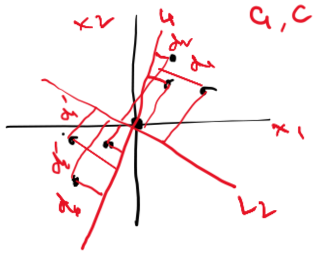
$X' = L_1 X_1 + L_2 X_2 + L_3 X_3 + L_4 X_4$

Principal Component Analysis :- (PCA)

X_1 X_2



Step1: Find centroid of whole data
Step2: Shift the origin to the center
Step3: Try to fit a line that separates as much as possible the components
↳ This line is PC1
Step4: Use PC1 to rescale the data and find PC2; which is perpendicular to PC1
↳ orthogonal



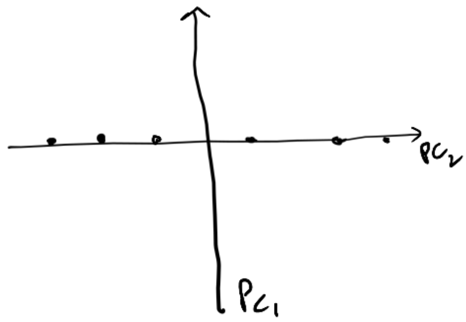
c_1, c_2 - Components

For L_1, find $\sum_{i=1}^{n} d_i^2 = D_{L_1}$

For L_2 find $\sum_{i=1}^{n} d_i^2 = D_{L_2}$

} Choose Components such that D_{L_1} & D_{L_2} are maximized!

→ For our example we had 2 features x_1 & x_2
↳ 2 component lines PC_1 & PC_2 were created



→ We had 3 features: x_1, x_2, x_3
↳ PC_1, PC_2, PC_3

→ Principal Components are linear combinations of the original features.

$x+a, y+b$
lin comb.

→ We have p-features → Use $PC_1, PC_2, \dots, PC_p$ → p Principal Components
↳ Start with PC_1 & PC_2

Q) How much of the variability of the target variable is captured?

PC_1, PC_2 → 78%
PC_1, PC_2, PC_3 → 84%
$PC_1, PC_2 \dots P_4$ → 91%
⋮
$PC_1, PC_2 \dots PC_7$ → 98% ←
$PC_1, PC_2 \dots PC_8$ → 98.3%
$PC_1 \dots PC_9$ → 98.5%

Threshold where we have satisfactory level of variability captured!

$PC_1 \rightarrow a_1X_1 + b_1X_2 + c_1X_3 \dots p_1X_p$
$PC_2 \rightarrow a_2X_1 + b_2X_2 + c_2X_3 \dots p_2X_p$

Q) How much info of the original features is captured?

Q) How to treat imbalanced data? (Imbalanced Target variable)

1 80 90 75
2 20 10 25

70 Y → 70 Y
30 N → 60 N } ⇒ Creating a bias

→ Resampling → Oversampling of minority class

→ Undersampling of majority class — 70 Y, 30 N → 30 Y, 30 N } ⇒ Over sampling

→ SMOTE (Synthetic Minority oversampling Technique)

SMOTE → Creates synthetic samples by interpolating b/w existing samples

↳ For each minority sample → K nearest neighbours (K=5)
↳ from same class.

↳ Select 1 or more samples from this at random

↳ Create the new synthetic point

$x_1 \longrightarrow$ Synthetic point $= x_1 + \lambda(x_{NN} - x_1)$ → Difference is the gap b/w the points
↳ Nearest neighbour point)

→ Use class weights :- Hyperparameter named "class_weight"
↳ By default "balanced"
↳ Most ensemble tech: Xgboost, RF.

→ These steps should also come before splitting the train & test dataframes.

——————— × ———————